UNSW COMP9900 Information Technology Project

Final Project Report

MentorPlan: AI-Mediated Teacher Decision Support for Data-Informed Lesson Planning

Project Information

Project field	Value
Project number	Project 25
Team	F15C-BREAD
Client	Sara Mashayekh (sara.mashayekh@unsw.edu.au)
Submission	9 August 2026
Document status	Final project report

Team Members

Name	Email	Student ID	Official role
Yulong Jiang	z5485481@ad.unsw.edu.au	z5485481	Scrum Master
Qianyi Zhang	z5623911@ad.unsw.edu.au	z5623911	Product Owner
Xingqian She	z5561648@ad.unsw.edu.au	z5561648	Developer
Huaijin Chen	z5559709@ad.unsw.edu.au	z5559709	Developer
Yi Zhu	z5529745@ad.unsw.edu.au	z5529745	Developer
Yihong Yao	z5593964@ad.unsw.edu.au	z5593964	Developer

Table of Contents

0. Project Overview 3
0.1 Target User Story 3
0.2 Final Scope 3
1. Installation Manual 3
1.1 Prerequisites and Configuration 4
1.2 Start and Verify 4
1.3 Production Deployment Option 5
2. System Architecture Diagram 5
2.1 Presentation and Access Layer 5
2.2 API, Services and Persistence 6
2.3 End-to-End Data Flow 6
3. Design Justifications 7
3.1 Adaptive Support Without Changing Pedagogy 7
3.2 From One Card to Four Decision Objects 7
3.3 Deterministic Analytics Instead of Opaque Clustering 8
3.4 Data Cleaning and Learning Analytics Pipeline 8
3.4.1 Data Input and Processing 8
3.4.2 Analytics and Student Privacy in Analytics Pipeline 9
3.5 Privacy Filter and Role-Based Access 9
3.6 Controlled Lesson-Plan Generation 9
4. User-Driven Evaluation of the Solution 9
4.1 Main Implemented Features 10
4.2 Evaluation Criteria and Method 10
4.3 Current Evidence and Fair Assessment 11
4.4 Verification Results and Residual Gaps 12
5. Limitations and Future Work 12
5.1 Client Handover 13
References 14
Appendix A. Verification and Evidence Index 15

0. Project Overview

Responsible member: *Member 1 - Frontend and Report Integration |* ***Supporting:*** *Member 5 validates pedagogy; Product Owner validates client wording*

MentorPlan is a web-based teacher decision-support system developed for client Sara Mashayekh. Its primary users are pre-service and early-career teachers who need help translating assessment evidence into lesson-planning decisions. The system does not attempt to replace teacher judgement. Instead, it makes the link between data evidence, pedagogical reasoning, teacher decisions and the final lesson plan visible.

This goal also aligns with the Australian Professional Standards for Teachers, which require teachers to interpret assessment data and use it to evaluate and improve teaching programs (Australian Institute for Teaching and School Leadership [AITSL], 2011).

The design responds to a practical gap: learning analytics dashboards commonly describe past performance, while general generative-AI tools can produce lesson plans without making their reasoning or evidence trail sufficiently visible. MentorPlan combines structured lesson context, de-identified assessment analytics, recommendation cards, explicit teacher decisions, editable lesson plans and supervision feedback within one workflow (Mandinach & Gummer, 2016; UNESCO, 2023).

0.1 Target User Story

Responsible member: *Member 1 - Frontend and Report Integration |* ***Supporting:*** *Members 3, 5 and 6 validate system responses*

Ms Lee, an early-career Year 7 Science teacher, signs in and chooses the level of AI mentoring she prefers. She records an Ecosystems lesson context, uploads a de-identified assessment spreadsheet, and reviews class-level patterns. MentorPlan generates four recommendation cards for teaching activity, grouping, differentiation and formative assessment. Ms Lee accepts, skips or modifies each card, records her reflection, and generates a plan only from approved or teacher-modified decisions. She edits the draft, saves a new version, exports it, and receives feedback from an assigned experienced teacher.

0.2 Final Scope

Responsible member: *Member 4 - Backend and Architecture |* ***Supporting:*** *Members 1, 2, 3, 5 and 6 confirm their feature boundaries*

In scope: user authentication, role-based access, teacher support profile, lesson context, CSV/XLSX/XLS upload and preview, column mapping, learning analytics, recommendation cards, teacher decisions, lesson-plan generation, editing, version history, DOCX/PDF export, and supervisor feedback.

Privacy boundary: raw student rows, names, aliases, student identifiers and local storage paths are blocked from external-AI payloads; raw previews remain owner-only.

Out of scope: live school/LMS integration, formal model fine-tuning, unrestricted processing of identifiable student data, and autonomous lesson-plan approval without teacher review.

The report is based on repository commit 91d42ce, which integrates the final frontend, backend, AI, data-processing and student-pseudonymisation work available on main on 5 August 2026.

1. Installation Manual

Responsible member: *Member 4 - Backend, Security and DevOps |* ***Supporting:*** *Member 1 verifies the browser URL; Member 6 verifies AI settings*

The development stack is Dockerised and includes PostgreSQL, pgAdmin, a FastAPI backend and a React/Vite frontend. The assessor does not need to install Python, Node.js or PostgreSQL directly.

1.1 Prerequisites and Configuration

Responsible member: *Member 4 - Backend, Security and DevOps |* ***Supporting:*** *Member 6 confirms provider variables*

1. Install and start Docker Desktop with Docker Compose support.

2. Open PowerShell in the repository root containing docker-compose.yml.

3. Copy .env.example to .env. Do not commit .env or any real API key.

4. Set JWT_SECRET_KEY to a random value of at least 32 characters and set the bootstrap administrator email, password and display name.

5. For a no-cost local demonstration, keep REQUIRE_EXTERNAL_AI=false; the backend will use deterministic rule-based fallbacks. For external generation, configure the relevant provider key without adding it to Git.

Variable	Requirement	Purpose
JWT_SECRET_KEY	Required; at least 32 characters	Signs HS256 access tokens.
ACCESS_TOKEN_EXPIRE_MINUTES	Optional; default 60	Controls access-token lifetime.
BOOTSTRAP_ADMIN_EMAIL	Recommended	Creates the first administrator if absent.
BOOTSTRAP_ADMIN_PASSWORD	Recommended; strong secret	Password for the bootstrap administrator.
CORS_ALLOWED_ORIGINS	Optional for local stack	Allowed browser origins.
REQUIRE_EXTERNAL_AI	false for local fallback	Fails startup in production when AI configuration is required but missing.
AI_PROVIDER / AI_API_KEY	OpenAI recommendation path	External recommendation-card generation; current adapter supports OpenAI-compatible use.
OPENAI/GEMINI/ANTHROPIC/QWEN keys	Optional	Lesson-plan provider pool; configured providers are tried in order.

1.2 Start and Verify

Responsible member: *Member 4 - Backend, Security and DevOps |* ***Supporting:*** *Member 1 confirms the complete UI loads*

1. Run: docker compose up –build

2. Wait until the backend and database are healthy and the frontend build is serving requests.

3. Open `http://localhost:3000` for the application.

4. Open `http://localhost:8000/docs` for interactive API documentation and `http://localhost:5050` for pgAdmin if database inspection is required.

5. Check `http://localhost:8000/api/v1/health` and /api/v1/health/database before testing the workflow.

6. Stop the stack with docker compose down. Do not add the -v flag unless intentionally deleting persisted database and export volumes.

Verification note: frontend production compilation, backend compilation and focused data-processing checks passed on 5 August 2026. A fresh Docker end-to-end run could not be repeated during report assembly because Docker Desktop was not running; this limitation is disclosed in Section 4.3.

1.3 Production Deployment Option

Responsible member: *Member 4 - Backend, Security and DevOps |* ***Supporting:*** *Product Owner confirms client handover domain*

The repository also contains docker-compose.prod.yml, a production frontend image and a Caddy gateway. On a Linux host with a public domain, the stack can terminate HTTPS, proxy /api/v1 to FastAPI, serve the built frontend and persist PostgreSQL, upload, export and certificate data in named volumes. This is deployment capability, not proof that a stable public environment has already been handed over.

2. System Architecture Diagram

Responsible member: *Member 4 - Backend and Architecture |* ***Supporting:*** *Members 1-3 and 5-6 validate component names and arrows*

The final system is a role-aware web application with a React presentation layer, a versioned FastAPI REST API, service and repository boundaries, PostgreSQL persistence, local or Docker-volume file storage, and optional external-AI providers. The architecture diagram labels data flows rather than listing technologies alone.

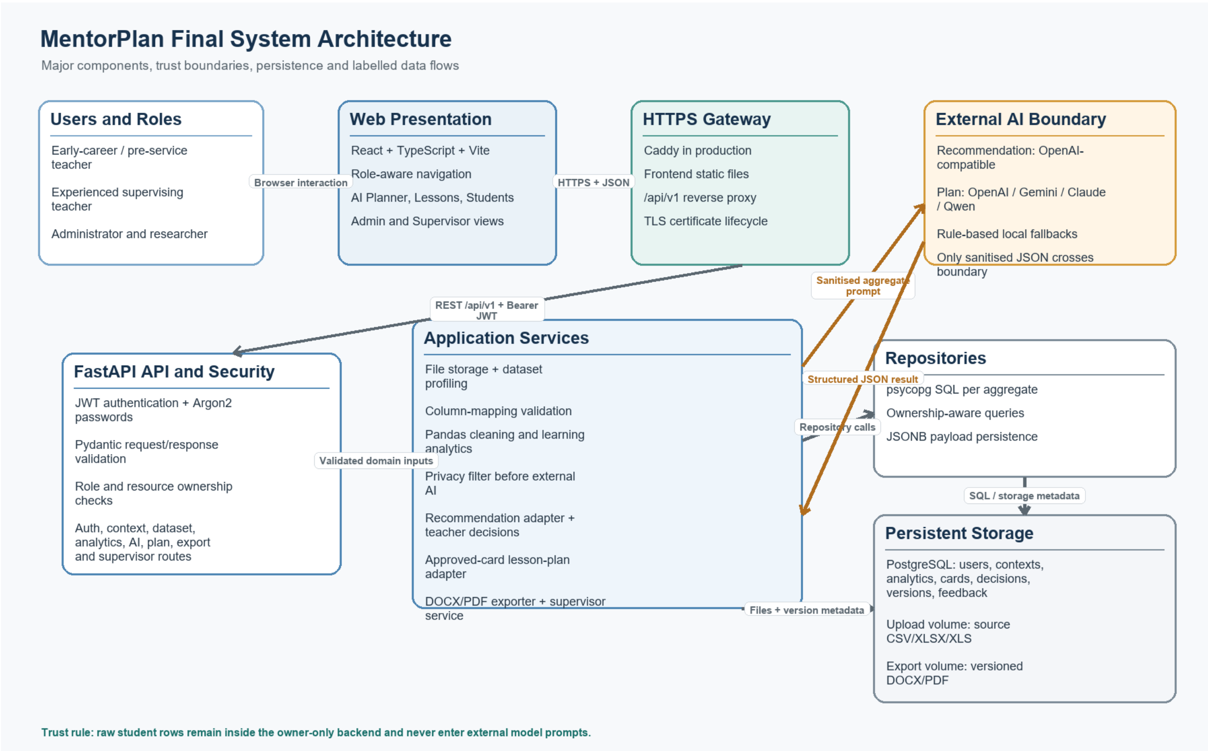


Figure 1. Final MentorPlan architecture. External models receive sanitised lesson context, aggregate analytics and approved decision inputs, not raw student rows.

2.1 Presentation and Access Layer

Responsible member: *Member 1 - Frontend and Integration |* ***Supporting:*** *Member 4 validates role rules*

The React/Vite single-page application presents separate navigation based on the authenticated role. Teachers access Dashboard, Lessons, Students, AI Planner and Account. Experienced teachers gain Supervisor access only when an active supervision assignment exists. Researchers receive read-only supervisor audit access, while administrators manage users, supervision assignments and audit views.

2.2 API, Services and Persistence

Responsible member: *Member 4 - Backend and Architecture |* ***Supporting:*** *Members 2, 3, 5 and 6 confirm service boundaries*

FastAPI exposes /api/v1 routes for authentication, users, support profiles, lesson contexts, datasets, mappings, analytics, recommendations, decisions, lesson plans, exports and supervision. Pydantic schemas validate requests and responses. Dependencies enforce bearer-token authentication, role checks and teacher ownership before route functions delegate business rules to services. Repositories centralise psycopg SQL and store structured or evolving payloads in PostgreSQL JSONB columns.

Uploaded files and generated exports are stored outside relational rows and referenced by storage URIs. This keeps large binary files out of PostgreSQL while retaining ownership and audit metadata. In production, named Docker volumes persist uploads, exports and database state.

2.3 End-to-End Data Flow

Responsible member: *Member 4 - Backend and Architecture |* ***Supporting:*** *All technical members validate their stage*

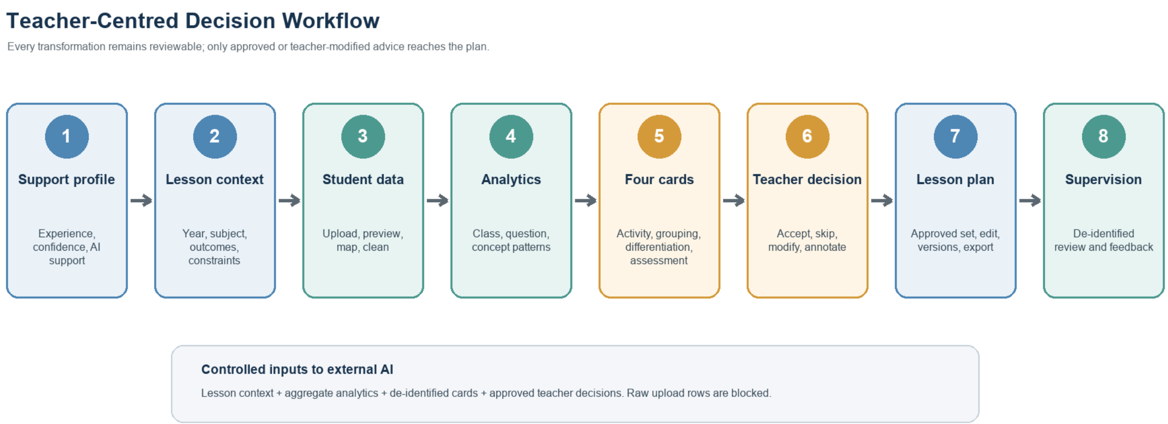


Figure 2. Teacher-centred workflow and controlled data flow from support profile to supervision feedback.

1. A user registers or signs in. Passwords are Argon2-hashed and FastAPI returns a time-limited JWT.

2. The teacher saves a support profile and lesson context. The profile changes explanation and reflection scaffolding, not the educational recommendation itself.

3. The teacher uploads CSV/XLSX/XLS data. The backend stores the file, profiles tables and exposes owner-only previews.

4. Column mappings and analytics produce class, question and concept summaries plus possible support and extension patterns.

5. The privacy filter removes direct identifiers and raw rows before any external-AI call. Recommendation generation produces four typed cards with evidence, reasoning and alignment fields.

6. Teacher decisions are persisted. Skipped cards are excluded; modified cards replace their original content in the effective approved set.

7. Lesson-plan generation combines lesson context, analytics and approved or modified recommendations. Each teacher edit creates a new version.

8. DOCX/PDF exporters render a chosen version. Assigned experienced teachers review de-identified workflows and publish version-bound feedback.

3. Design Justifications

Responsible member: *Member 5 - AI Recommendations (section lead) |* ***Supporting:*** *Members 2-4 and 6 own their technical subsections*

The strongest justification is not a list of chosen technologies; it is the sequence from an initial design, through client or engineering feedback, to a changed implementation and a reason that the final design is more effective. The table below summarises that evolution.

Stage	Design at that stage	Why it changed
Initial proposal	Broad end-to-end mentor with lesson context, upload, analytics, recommendations, decisions and export; some architecture material anticipated K-means and a generic multi-provider gateway.	The proposal expressed the desired scope but several components were not connected or implemented.
Demo A	A basic upload/analytics path and one recommendation card could feed lesson-plan generation.	Teacher control was weak: no multi-card comparison and no explicit accepted/skipped/modified decision set.
Demo B	Added support profile, four card types, accept/skip/modify/annotate actions, reflection prompts, approved-card plan generation, editing, version history and export.	The workflow became human-in-the-loop and traceable rather than direct AI-to-plan generation.
Final client iteration	Client requested generated output for inspection, a supervising experienced-teacher view, feedback capability and a live link.	Added authentication/RBAC, admin supervision assignment, de-identified supervisor workflow, version-bound feedback and a production Docker/Caddy stack.

The final iteration directly addressed the client meeting of 28 July 2026: the client asked to inspect generated output, requested an experienced-teacher view that could see how and why a novice teacher planned, and asked for feedback and an independently testable link. Authentication, supervision assignment, workflow drill-down, feedback and production deployment files were added in response.

3.1 Adaptive Support Without Changing Pedagogy

Responsible member: *Member 5 - AI Recommendations |* ***Supporting:*** *Member 1 supplies support-profile UI evidence*

The initial workflow treated every teacher identically. Client requirements later introduced three profile questions: teaching experience, planning confidence and preferred AI support level. The final implementation snapshots these values into the lesson context and uses them to adjust mentoring detail, reflection scaffolding and explanatory language. It explicitly records an adaptation boundary: profile answers do not change the educational recommendation. This avoids giving different pedagogical advice merely because one teacher reports lower confidence, while still supporting cognitive apprenticeship through coaching and reflection (Collins et al., 1986).

3.2 From One Card to Four Decision Objects

Responsible member: *Member 5 - AI Recommendations |* ***Supporting:*** *Member 6 validates approved-set behaviour*

Demo A passed one recommendation directly into plan generation. The final design creates a generation batch containing teaching_activity, grouping, differentiation and formative_assessment cards. Each card carries evidence, reasoning, a recommended action, lesson/data/assessment alignment and an adaptive reflection prompt. Teachers can accept, skip, modify or annotate cards. The effective decision state is reconstructed from persisted decision records, and plan generation receives only approved or teacher-modified cards. This change increases teacher agency and makes the provenance of final plan content auditable.

3.3 Deterministic Analytics Instead of Opaque Clustering

Responsible member: *Member 3 - Learning Analytics |* ***Supporting:*** *Member 2 validates cleaned input assumptions*

The proposal considered K-means for grouping. The implemented analytics instead uses transparent thresholds: below 0.50 for intensive support, 0.50 to below 0.70 for targeted support, 0.70 to below 0.85 for core practice, and at least 0.85 for extension. Similar-needs groups use balanced chunks; mixed-readiness groups use a deterministic snake allocation. Every output is labelled as a suggestion requiring teacher review. This was a deliberate trade-off: deterministic rules are easier to explain, test and audit with a small prototype dataset than an unsupervised cluster whose meaning can shift between runs.

3.4 Data Cleaning and Learning Analytics Pipeline

Responsible member: *Member 2 - Data Ingestion |* ***Supporting:*** *Member 3 owns analytics calculations*

Since different teacher may upload different assessment data. For example, Some files use one column for each question and other files put every answer in a new row. Column names may also be different. So, the raw data cannot be directly used by learning analytics engine. In order to solve this, we create the data processing part which is between the uploaded file and other functions. The data processing part is not only for data cleaning but also making the later result has same meaning by standardizes input lines into a uniform long format.

3.4.1 Data Input and Processing

The current workflow of the data processing part supports CSV, XLSX, and XLS files. Excel documents may contain several non-empty sheets. Each sheet is processed individually. CSV files may have some problems like delimiter or text encoding. We check the problems when reading, empty rows and columns will be removed, and if blank or duplicate headers are detected, a stable column name will be assigned and log a warning.

There are two common data forms: wide data(where each row represents a student and columns represent question scores) and long data(where each row represents a single answer). Input data mainly arrives in one of two forms. Files only contain summary are also acceptable.

In order to handle various files with different column names across different schools and platforms, this component tries to check the column names against a predefined set. However, this check may fail. In this case, teachers can save a column mapping. The system uses this mapping first. Mapping the rigth column is important because if the mapping is wrong, the score meaning can be wrong too.

After mapping, every valid answer is transformed into one common form denoted as a canonical response. This structure stores the student, assessment, question, score and maximum score. It may also store the concept. By using the structure, the learning analytics engine only needs to handle with one data form, which means wide and long data can be processed by the same analytical pipeline. If an uploaded dataset contain both a wide and a long table, in this case, we use the long table.

Each score needs a maximum score. For boolean (0/1) questions, the maximum score can be 1. For other wide format questions, this value is from the file or the mapping. We do not use the highest student score, since there is no student get the full score in the class. If we can not find the value, the system stops.

There is another problem. If the table structure is entirely unrecognizable, or if a selected table/mapping is missing, the system terminates the analysis. However, one bad row is different. Rows with no student ID or a wrong score will be removed. Duplicate records with different scores will be deleted. The system will also save the number of deleted records and the reason for deletion. Then, other rows can still be used.

3.4.2 Analytics and Student Privacy in Analytics Pipeline

The canonical responses will be used by the system to aggregates performance metrics at the class, question, and concept levels. This allows the system to highlight areas of strength and weakness, and dynamically construct student groupings.

In the case that we only have summary data, we can still make some class level or question level insights by using the summary data. However, we cannot make student groups because there is no student level data. If there is no question explanations, the result may have less information.

Before analytics, the system changes the student IDs to `STU-xxx`. The learning analytics engine uses these new IDs. After get the result, the original ID can be put back.

Some limits. We have code some functions to read table PDFs. For future, it may be able to accept PDF, but in current version, PDFs are not acceptable.

3.5 Privacy Filter and Role-Based Access

Responsible member: *Member 4 - Backend and Security |* ***Supporting:*** *Member 5 validates AI payload filtering*

Privacy is enforced at two layers. First, JWT authentication, ownership checks and role rules prevent one teacher from reading another teacher's raw uploads. Second, sanitize_for_external_ai recursively removes keys such as student_id, student_ref, alias, raw_rows and storage_uri, and redacts email, phone and serialised student identifiers inside strings. The filter returns a non-sensitive report recording removed paths and confirms that raw student data was not sent. This is stronger than relying on the frontend not to display sensitive fields, because the control sits at the backend boundary immediately before external model calls.

3.6 Controlled Lesson-Plan Generation

Responsible member: *Member 6 - AI Lesson Plan |* ***Supporting:*** *Member 5 validates source recommendation data*

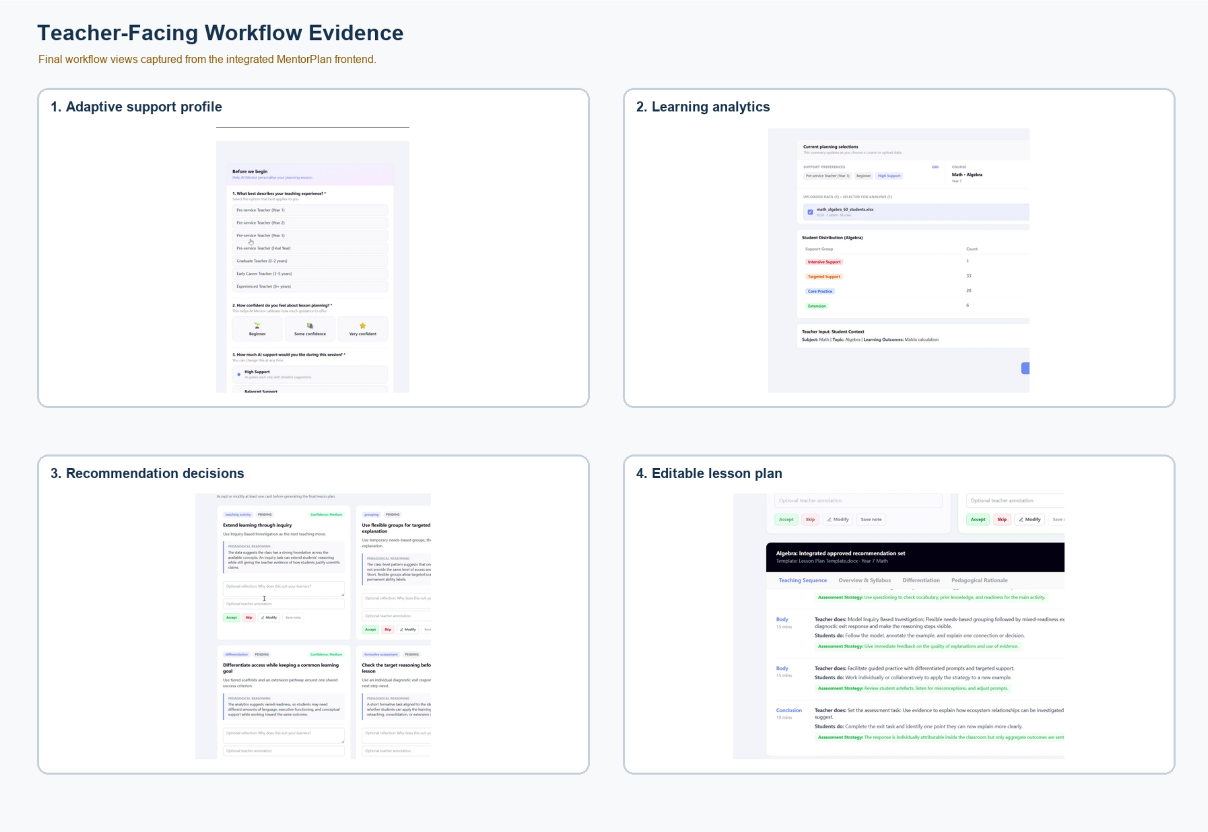
The final adapter aggregates distinct approved cards into one stable plan-generation input. It preserves source-card types, implementation steps, differentiation supports, assessment suggestions and teacher decision context. The plan schema is normalised against a rule-based fallback and validated before persistence. Lesson-plan generation can try configured providers in the order OpenAI, Gemini, Claude and Qwen; if no provider is configured and external AI is not required, the local fallback still produces a structurally valid demonstration plan. The recommendation-card adapter itself supports OpenAI-compatible external generation plus a rule-based fallback. Teacher edits never overwrite an earlier version: each save creates a new lesson_plan_versions row and exports target a specific version.

4. User-Driven Evaluation of the Solution

Responsible member: *Member 1 - User Journey and Evaluation Integration |* ***Supporting:*** *Members 2-6 provide criterion-specific evidence*

Evaluation is organised around whether target users can make a defensible planning decision, not merely whether individual endpoints return 200 responses. The principal scenario uses the client-provided Year 7 Science assessment package and compares expected data patterns, teacher decisions and plan output across one coherent workflow.

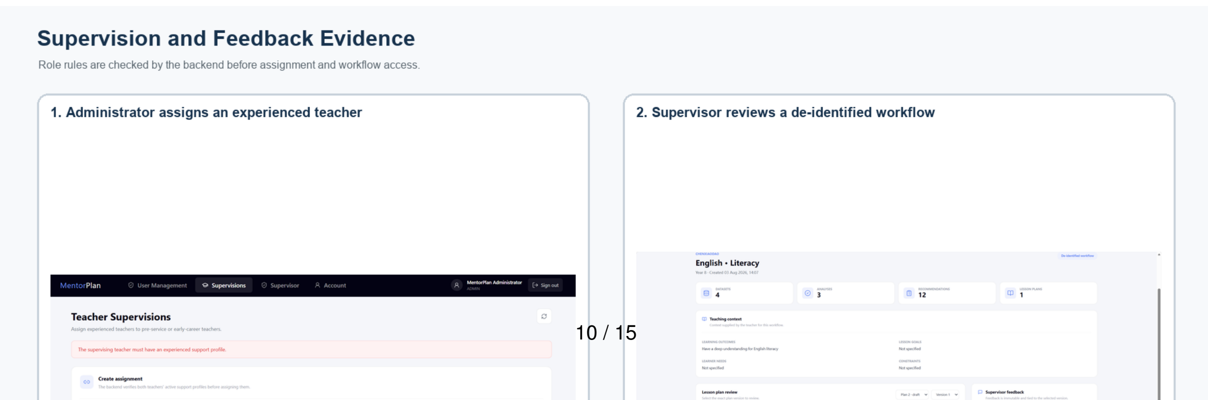
Figure 3. Teacher-facing evidence from the final implementation: adaptive profile, analytics, recommendation decisions and editable lesson plan.



4.1 Main Implemented Features

Responsible member: *Member 1 - Frontend and Integration |* ***Supporting:*** *Technical owner listed in each row*

Feature	User-visible outcome
Authentication and roles	Register/login, teacher ownership, admin user management, researcher/admin audit access
Adaptive support profile	Experience, confidence and preferred support adjust explanation/reflection support
Lesson context	Year, subject, topic, outcomes, goals, needs, constraints and teacher interpretation
Student data	CSV/XLSX/XLS upload, profile, preview, stored mappings and analytics trigger
Learning analytics	Class/question/concept summaries, evidence, support levels and grouping suggestions
Recommendation cards	Four typed cards with visible evidence, reasoning, alignment and reflection prompts
Teacher decisions	Accept, skip, modify and annotate; effective approved state is persisted
Lesson plan	Generate from approved/modified cards, edit, save versions and export DOCX/PDF
Supervision	Admin assignment, de-identified workflow review and version-bound experienced-teacher feedback



Criterion	Success definition	Measure
E1 Task completion	A new teacher completes sign-in to export without developer intervention.	Completion rate, failed steps, elapsed time
E2 Data correctness	Reported class/question/concept values match expected values.	Absolute error, recognised rows, exclusion reasons
E3 Explainability and agency	A teacher can identify evidence/reasoning and control which advice reaches the plan.	Card completeness, decision persistence, approved-set trace
E4 Privacy and access control	Raw/identifiable data is not sent externally or exposed across users.	Blocked fields, ownership tests, role tests
E5 Plan usefulness	The exported plan is concise, aligned, editable and usable for the stated lesson.	Client rubric, template coverage, revision count
E6 Operability and handover	A fresh host can run the system and the client can access a stable link and documentation.	Fresh-install success, uptime check, handover confirmation

4.3 Current Evidence and Fair Assessment

Responsible member: *Member 3 - Evidence Quality |* ***Supporting:*** *Members 1, 2, 4, 5 and 6 update their row*

Criterion	Current status	Evidence-based finding	Required before submission
E1	Partially verified	The final demonstration recording shows the teacher workflow through plan generation and export. Backend compile and frontend production build passed. The supplied workflow test covers the remaining APIs but inserts aggregate analytics rather than exercising the upload route.	A clean-host timed usability run remains future evaluation work.
E2	Focused checks passed	Twelve self-contained data-processing checks passed, and the grouping/strength check passed against the client gold-standard workbook on 5 August 2026. The full script still expects a repository /data fixture that is not packaged in main.	Package the complete fixture set and automate the full suite in CI.
E3	Implemented with UI limitation	Decision state and effective modified content reload; approved cards are selected for generation. Saved reflection/annotation history is available through the API but is not repopulated into the editable input fields after refresh.	Hydrate the input fields from decision-state history.
E4	Implemented; runtime retest pending	JWT/RBAC, ownership dependencies, deterministic analytics pseudonymisation and recursive external-AI sanitisation are implemented. Integration scripts cover auth and supervision permissions, but Docker was unavailable during report assembly.	Run the integration scripts against the final production image before handover.
E5	Needs client evaluation	DOCX/PDF export and versioning exist, but the 28 July client explicitly requested the generated output for inspection against the supplied training/template material.	Send one final output, collect written feedback and evaluate against a fixed rubric.
E6	Deployment files exist; handover not proven	Production Compose, Caddy and deployment notes exist. A stable hosted client URL and final handover acknowledgement are not present in the supplied project evidence.	Deploy, run health checks, provide accounts/sample data and capture client acknowledgement.

4.4 Verification Results and Residual Gaps

Responsible member: *Member 3 - Learning Analytics |* ***Supporting:*** *Members 2 and 4 implement or verify the fix*

A previously identified mapping integration gap has been resolved in the current main branch: the analytics route loads dataset mapping configuration and forwards it through run_basic_analytics into analyse_assessment_package. The latest data-processing merge also pseudonymises student identifiers before analytics calculations. Current static and focused verification results are summarised below; they do not substitute for a final clean-database Docker run.

Verification	Result	Evidence
Repository baseline	PASS	main at 91d42ce; worktree clean before report generation
Backend syntax/import compilation	PASS	python -m compileall -q backend
Frontend production build	PASS	Vite 6.3.5; 1,650 modules transformed
Focused data processing	PASS	12 self-contained checks plus grouping/strength check on client workbook
Dependency audit	RISK	npm reported one moderate and two high vulnerabilities
Docker/API end-to-end	NOT RE-RUN	Docker Desktop daemon was unavailable during report assembly

5. Limitations and Future Work

Responsible member: *Member 6 - AI Lesson Plan and Handover |* ***Supporting:*** *Members 2-5 own technical limitation rows; Product Owner owns client evidence*

The prototype demonstrates a coherent teacher-controlled workflow, but it should not be represented as production-ready school infrastructure. The following limitations materially affect reliability, generalisability or handover readiness.

Limitation	User impact	Future work and resources
Evaluation uses synthetic and limited client examples	Correctness and usefulness may not generalise across subjects and schools.	Create a multi-subject benchmark and conduct 5-10 teacher task sessions; data preparation plus ethics/ client coordination, 2-3 weeks.
External-AI output remains variable	Provider availability, rate limits and model changes can alter plan quality.	Version prompts/models, add schema-retry telemetry and a regression set; one AI engineer, 1-2 weeks plus API budget.
Privacy filtering remains rule-based	Unexpected identifiers embedded in free text may escape fixed patterns.	Add configurable DLP/entity detection, adversarial privacy tests and retention controls; backend/ security effort, 2-4 weeks.
Reflection fields do not hydrate after refresh	Teacher reasoning is persisted but the UI does not fully support revisiting it.	Hydrate fields from decision history and add refresh tests; frontend plus backend, 1-2 days.
Canonical student responses are not persisted	Fine-grained longitudinal analysis and reproducible re-analysis are limited.	Persist validated canonical records with retention controls; backend plus data engineer, 1-2 weeks.
Stable public operations are unproven	Client access depends on a maintained host, domain, backups and monitoring.	Provision a managed host, domain, backups, alerts and an operational owner; 2-3 days setup plus recurring cost.
Frontend dependency audit reports vulnerabilities	Known package vulnerabilities may create avoidable operational risk.	Review npm audit output, update compatible packages and add automated dependency scanning; frontend engineer, 1-2 days.

Limitation	User impact	Future work and resources
No LMS or school-system integration	Teachers manually upload files and duplicate context.	Define a privacy-reviewed LMS integration and SSO roadmap; multi-sprint work with institutional access.

5.1 Client Handover

Responsible member: *Member 6 - Handover Coordinator |* ***Supporting:*** *Member 4 supplies deployment/admin material; Product Owner sends the message*

The repository contains environment examples, local and production Docker Compose files, API documentation, database schema notes, authentication notes and deployment instructions. During the 28 July meeting, the client requested the generated lesson-plan output and a live link that could be tested independently. A stable hosted environment and final acknowledgement were not present in the supplied evidence at report assembly time, so handover is reported as pending rather than falsely described as complete.

Package item	Contents	Status
Source and release	main commit 91d42ce and repository documentation	Ready
Runbook	Local/production installation, health checks, backup and shutdown	Ready in docs
Access	Stable HTTPS URL and separate role-based test accounts	Pending hosted environment
Test package	Client Year 7 Science workbook and focused expected-result checks	Available locally
User guide	Teacher workflow, decision meanings, supervision assignment and feedback	Partly covered by report and API docs
Known limitations	Evaluation, reflection reload, external-AI variability, operations and retention	Disclosed in Section 5
Communication evidence	Final handover message and client acknowledgement	Pending external communication

Planned final communication will include the hosted URL, role-based test accounts shared through a separate secure channel, sample workflow and data, generated output, installation/deployment documentation, known limitations and a support contact. The Product Owner will send the message and provide the tutor with the acknowledgement screenshot after completion, as permitted by the assessment specification for a pending handover.

References

Responsible member: *Member 1 - Report Integration |* ***Supporting:*** *Every member supplies complete source metadata for their claims*

Australian Institute for Teaching and School Leadership. (2011). Australian professional standards for teachers. AITSL.

Collins, A., Brown, J. S., & Newman, S. E. (1986). Cognitive apprenticeship: Teaching the craft of reading, writing, and mathematics (Technical Report No. 403). Center for the Study of Reading, University of Illinois at Urbana-Champaign.

Mandinach, E. B., & Gummer, E. S. (2016). What does it mean for teachers to be data literate: Laying out the skills, knowledge, and dispositions. Teaching and Teacher Education, 60, 366-376. https://doi.org/10.1016/j.tate.2016.07.011

UNESCO. (2023). Guidance for generative AI in education and research. https://doi.org/10.54675/EWZM9535

Appendix A. Verification and Evidence Index

Responsible member: *Member 1 - Report Integration |* ***Supporting:*** *All members sign off*

Evidence area	**Primary code/document paths**
Docker and deployment	docker-compose.yml; docker-compose.prod.yml; docs/production-deployment.md
Authentication/RBAC	backend/app/api/routes/auth.py; dependencies/auth.py; docs/user-auth.md
Data pipeline	backend/app/data_processing/*; scripts/test_data_processing.py
Mapping integration	routes/analytics.py; column_mappings_repository.py; services/analytics_adapter.py
Student pseudonymisation	data_processing/student_identity.py; pipeline.py; privacy_filter.py
Recommendations and decisions	recommendation_adapter.py; recommendations.py; teacher_decisions.py
Plans and exports	lesson_plan_adapter.py; lesson_plan_exporter.py; lesson_plans.py; exports.py
Supervisor workflow	admin_supervisions.py; supervisor.py; supervisor_service.py; frontend supervisor components
Integrated workflow	backend/scripts/test_completed_workflow_api.py; note synthetic analytics fixture